This class is small enough that I can, thankfully, give the midterm in our lecture classroom. I am worried that this will not always be true for my *undergraduate* classes. Use dynamic programming to help me solve the following problem, which I am sure to have in a few years. In the future, I will be preparing for the 2030-31 school year. UCI is going to allow me to prepare an old building to be used for giving exams to students. There are n rooms in this building, numbered $1 \dots n$. Room i can be used to give $S[i]$ students an exam. I will use a subset of the rooms for exams. I want to host as many students at the same time as I can.

Unfortunately, there have been several incidents of students cheating at other universities who tried a similar thing. The students cheated by tapping secret codes through walls. To prevent this type of cheating, I can use two adjacent rooms only if I demolish the wall between them. Irvine's chief architect has determined that demolishing the walls on both sides of the same room would threaten the building's structural integrity. For this reason, I can never host students in three consecutive rooms

Use **dynamic programming** to describe an efficient algorithm that computes the largest number of students that I can host for testing without using three consecutive rooms. The input to your algorithm is an array $S[1 \dots n]$, where each $S[i]$ is the (non-negative integer) number of students that can fit in room i.

There is minimal, possibly zero, partial credit for a solution that solves the "no adjacent rooms" problem.

Caution: be very careful with indices in your code. Think carefully about what the recursive call(s) are intended to compute.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.